T22 Outils Tiers

Technique Intermediate

Les outils complémentaires indispensables de l'écosystème

Tracking des Coûts

Les outils de cette fiche sont des couches adjacentes: ils observent, configurent ou coordonnent des sessions. Ils ne remplacent pas le runtime qui possède la boucle agentique. Pour choisir un runtime ou un orchestrateur, consulter le Agent Harness Map.

ccusage : lit les enregistrements locaux d'usage des sessions et estime les coûts. Vérifier son comportement réseau et sa source de tarifs ; une entrée locale ne prouve pas l'absence d'accès réseau.

```
npx ccusage daily --last 1 # Dernière journée
ccusage daily       # Rapport journalier
ccusage blocks --live # Monitoring temps réel (fenêtres 5h)
ccusage daily --breakdown # Détail par modèle
```

ccburn : alternative Python avec graphiques de burn-rate en terminal. Utile pour visualiser la consommation relative aux limites de facturation, dans un ecosystème Python.

```
pip install ccburn
ccburn
```

Dashboard & monitoring

ccboard : TUI et interface web pour monitorer sessions, coûts et statistiques en temps réel.

```
cargo install ccboard
ccboard         # Interface TUI
ccboard --web   # UI web sur localhost:3000
```

Partage social & Classement

Straude : dashboard social pour tracker et partager ses stats de consommation. Pousse les métriques quotidiennes (coût, tokens, modèles) vers un leaderboard public. Attention : envoie le hostname machine et un UUID device aux serveurs Straude. Utiliser `--dry-run` avant le premier push.

```
npx straude@latest   # Auth + sync en une commande
straude push --dry-run  # Prévisualiser sans envoyer
straude status      # Streak, rank global, totaux
```

viberank : leaderboard couvrant plus d'outils (Claude Code, Codex, Gemini CLI, OpenCode, et d'autres). Envoie uniquement des totaux agrégés, ni hostname ni UUID device. Publie son jeu de données via une API JSON gratuite sous licence CC BY 4.0.

```
npx viberank-cli     # Soumettre les données ccusage
```

Optimisation tokens

RTK (Rust Token Killer) : proxy CLI qui filtre les outputs avant qu'ils entrent dans le contexte. Les économies dépendent de la commande et de sa sortie ; les estimations ne prouvent pas une réduction équivalente de la facture API. Voir fiche T20 pour le détail.

```
brew install rtk
rtk init          # Setup avec hook automatique
```

Orchestration Parallèle

Conductor : application macOS pour orchestrer plusieurs instances Claude Code en parallèle via git worktrees. Interface intégrée : diff viewer, CI/GitHub Actions, Linear, PR workflow. Installer depuis conductor.build (macOS uniquement en mars 2026).

Flux typique : créer un workspace par feature depuis une issue GitHub ou Linear, laisser les agents tourner, utiliser « Next Workspace » pour traiter uniquement ceux en attente d'input.

Tableau de Recommandation

Profil	Outils recommandés
Développeur solo	ccusage + claude-code-viewer
Petite équipe	ccusage + Conductor ou multiclaude
Multi-agents	Conductor ou Toad
Écosystème Python	ccburn + Claude Chic

Lacunes connues

Pas d'éditeur visuel pour les skills `.claude/skills/`. Pas de dashboard unifié combinant config, sessions, coûts et MCP. Un TUI dédié existe pour les serveurs MCP (`claude-code-config` , `pip install claude-code-config`), mais reste limité au scope `~/.claude.json` : ni `settings.json` , ni hooks, ni commandes.

Les capacités et la maintenance de ces projets évoluent séparément de Claude Code. Vérifier le dépôt, les permissions et la release ciblée avant installation; le glossaire distingue runtime, orchestrateur, protocole et outil adjacent.